Autoreason: When Iterative LLM Refinement Works

shl0ms

https://github.com/NousResearch/autoreason

ABSTRACT. Self-refinement is unreliable in general. We identify the conditions under which it works and present autoreason, a method that exploits those conditions. Each iteration produces three competing versions (the incumbent, an adversarial revision, and a synthesis), evaluated by a blind judge panel using Borda count aggregation. With claude-sonnet-4 across 5 subjective tasks, autoreason ranked first or second in every 7-judge blind comparison, winning 3/5 outright. With the stronger Sonnet 4.6, the method fails on open-ended tasks but wins on scope-constrained tasks, and eight remedy experiments identify scope as the root cause.

We validate these findings in competitive programming (81 CodeContests problems across 3 difficulty tiers). With matched compute budgets (6 LLM calls per problem), autoreason achieves 65% private-test pass rate vs. 58% for both single-pass sampling and standard critique-and-revise. Crucially, critique-and-revise overfits to visible test cases: it matches autoreason on public tests (79% vs. 85%) but fails to generalize to hidden tests. Autoreason's structured reasoning protects against this overfitting, with the strongest gains on hard problems (47% vs. 37% private pass rate). The method works when three conditions hold: constrained scope, external verification signal, and structured reasoning that goes beyond pattern-matching on feedback.

1 INTRODUCTION

Iterative LLM self-refinement is widely assumed to improve outputs, but the evidence is mixed. Self-Refine [4] demonstrates that critique-and-revise loops can help but provides no mechanism to detect when revisions make things worse. Huang et al. [5] show LLMs cannot reliably self-correct without external feedback. SlopCodeBench [2] documents progressive degradation in iterative code generation. The question is not whether self-refinement can work, but *when* it works and what makes it fail.

We present autoreason, extending Karpathy's autoresearch [1] beyond objective metrics to domains where quality is subjective or multi-dimensional. Each iteration produces three competing versions: the incumbent (A), an adversarial revision (B), and a synthesis (AB), evaluated by a blind judge panel. The method constructs a fitness function through the evaluation process itself.

Our central finding is that self-refinement works reliably when three conditions hold: (1) **constrained scope**—the task has bounded improvement space; (2) **external verification**—feedback comes from outside the generating model's reasoning loop; and (3) **structured reasoning**—the refinement process explicitly analyzes failure modes rather than pattern-matching on feedback. We establish these conditions across two domains: subjective writing (5 tasks, 8 remedy experiments) and competitive programming (81 CodeContests problems). In the code domain, autoreason's structured reasoning prevents the overfitting to visible test cases that standard critique-and-revise exhibits, yielding a 7-point improvement in hidden-test generalization.

2 METHOD

Critic. A fresh agent identifies problems in the current incumbent A. Finds problems only; no fixes. Optionally receives ground-truth data for fact-checking (Section 7).

Author B. A fresh agent revises A based on the critique. Sees the task, A, and the critique. No drafting history.

Synthesizer. A fresh agent receives A and B with randomized labels. Produces AB by taking the strongest elements of each.

Judge Panel. 3 fresh agents rank A, AB, B via Borda count (3/2/1 points). Randomized labels and presentation order. Conservative tiebreak: incumbent wins ties. Convergence at k=2 consecutive A wins. We use 3 judges in-loop to balance per-pass cost against evaluation stability, and 7 judges for final blind comparisons to increase statistical power where cost is amortized over a single evaluation.

Every role is a fresh, isolated agent with no shared context. The original task prompt anchors all evaluation. We chose k=2 because k=1 led to premature termination on noisy judgments in early experiments, while k=2 provided a stable stopping point across all tested runs; k=3 would increase cost without meaningfully changing outcomes given the Elo plateau analysis (Section 4.5). Temperature was set to 0.8 for authors (encouraging diverse revisions) and 0.3 for judges (encouraging consistent evaluation); these values were chosen based on preliminary runs and not formally ablated. Exact prompts are provided in Appendix A.

2.1 Formal Framing

Let d_t denote the incumbent document at pass t. Each pass produces candidates $\{d_t, B(d_t), S(d_t, B(d_t))\}$ where B is the adversarial revision operator and S is the synthesis operator. A judge panel J defines a stochastic preference relation: $J(x, y) \in [0, 1]$ is the probability that x is ranked above y. The winner is selected by Borda aggregation over n judges:

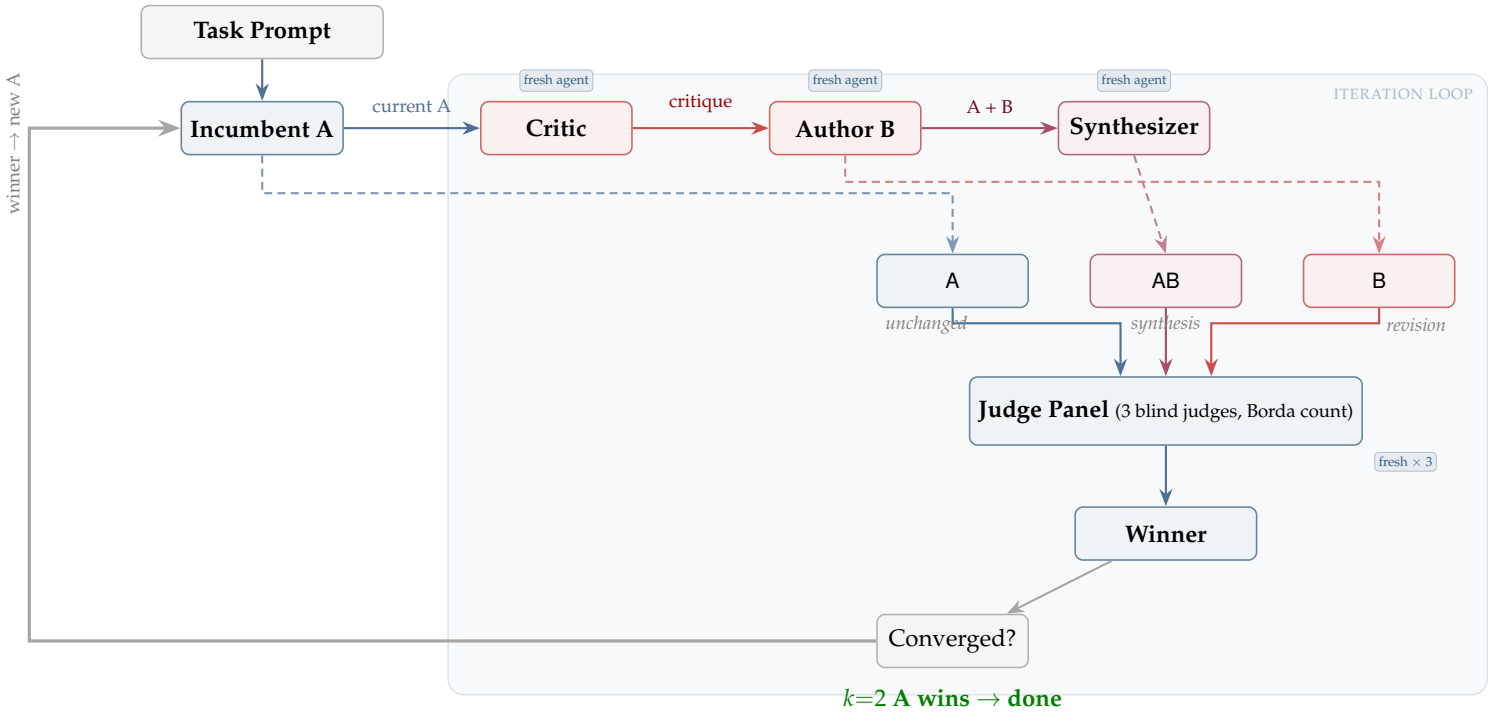

Figure 1. The autoreason loop. Each pass produces three candidates: the unchanged incumbent (A), a synthesis (AB), and an adversarial revision (B). A blind judge panel selects the winner via Borda count. If A survives k=2 consecutive passes, the loop terminates. Every role is a fresh, isolated agent.

$$d_{t+1} = \arg\max_{c \in \{A,B,AB\}} \sum_{i=1}^{n} (3 - r_i(c))$$

where $r_i(c)$ is judge i's rank for candidate c, with ties broken in favor of d_t.

This defines a Markov chain over document states. Convergence ($d_{t+1} = d_t$ for k consecutive passes) corresponds to an absorbing state. By the Condorcet jury theorem, if each judge independently identifies the better document with $p > 0.5$, panel accuracy increases with n. However, same-model judges share systematic biases, bounding effective independence below n.

The connection to PSRO [10] is loosely analogous rather than a direct equivalence: each pass adds a candidate to an empirical game, but autoreason lacks PSRO's full meta-strategy computation. Our retroactive analysis (Section 4.5) examines this framing empirically: preferences are near-transitive across 91 passes (1.1% Condorcet cycle rate), and Elo ratings plateau early, consistent with the Markov chain reaching its absorbing state.

3 Experiments

All experiments used claude-sonnet-4-20250514 (temp=0.8 authors, 0.3 judges, max_tokens=4096) unless otherwise noted. Five tasks: (1) go-to-market strategy, (2) notification system design, (3) remote work policy, (4) competitive positioning, (5) incident response. Each task ran autoreason to convergence (k=2) plus four baselines at 15 passes each from the same initial output, evaluated by a 7-judge blind panel. The paper-writing experiment (Section 7) used claude-opus-4. Scaling experiments (Section 5) used claude-sonnet-4-6.

Baselines. We compared against four iterative prompting strategies, each applied for 15 passes from the same initial document: (1) *Conservative*: "Make minimal improvements while preserving what works"; (2) *Improve this*: "Improve this document" with no further guidance; (3) *Harsh critic*: "Critically evaluate and rewrite, fixing all weaknesses"; (4) *Critique & revise*: a two-step loop where the model first produces a structured critique, then revises the document to address it (closest to Self-Refine [4]). All baselines use a single agent per pass with no judge panel or adversarial structure. Baseline prompts are provided in Appendix A.

Compute budget. Autoreason requires ∼6 LLM calls per pass (critic, author B, synthesizer, 3 judges) and typically runs 10–25 passes to convergence (∼60–150 calls per task), while each baseline requires ∼1 call per pass (∼15 calls total). This roughly 6× per-pass cost (and larger total cost due to more passes) is a real tradeoff; autoreason trades compute for evaluation quality.

4 Results

4.1 Single Pass vs. Iteration

Method	Calls	T1	T2	T3	T4	T5
Harsh critic	1	**30**	**31**	23	**30**	**31**
Critique & rev.	1	**30**	30	**31**	28	23
Autoreason	5	24	22	24	17	24
Improve this	1	14	14	18	23	20
Conservative	1	7	8	9	7	7

Table 1. Single-pass Borda scores (max 35). Bold indicates task winner. Autoreason places 3rd at 5× the cost.

Method	T1	T2	T3	T4	T5
Autoreason	**35**	19	**29**	25	**31**
Critique & rev.	13	**22**	26	**32**	19
Harsh critic	18	19	27	20	26
Conservative	21	19	15	18	15
Improve this	18	11	8	10	14

Table 2. Iterative Borda scores (max 35) at 15 passes. Autoreason wins tasks 1, 3, 5; critique-and-revise wins tasks 2, 4.

Single-pass autoreason averaged 22.2 Borda, placing 3rd on 4/5 tasks (Table 1). With iteration, it won 3 of 5 tasks (avg 27.8, rank 1.4), outperforming all single-pass methods (Table 2, Figure 2). The judge panel acts as a filter: revisions that score lower than the incumbent are rejected, while baselines accumulate drift unchecked. This filtering is defined by LLM preference, not an independent quality measure, so its effectiveness is bounded by the judges' biases. The scaling results (Section 5) demonstrate where this protection breaks down.

Autoreason excels on tasks with genuine tradeoffs (strategy, policy, incident response). Critique-and-revise wins on tasks with concrete technical requirements (system design, competitive positioning) where a direct find-and-fix loop is more efficient (Figure 3).

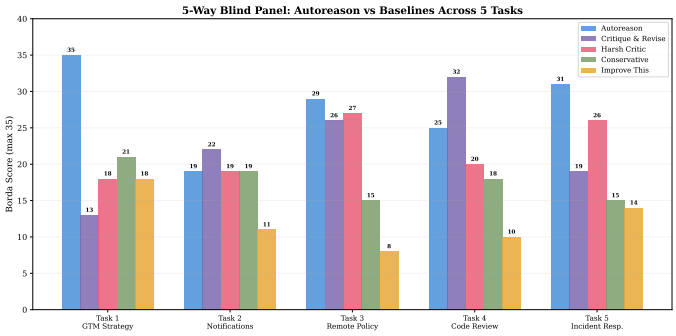

Figure 2. Per-task Borda scores at 15 iterative passes.

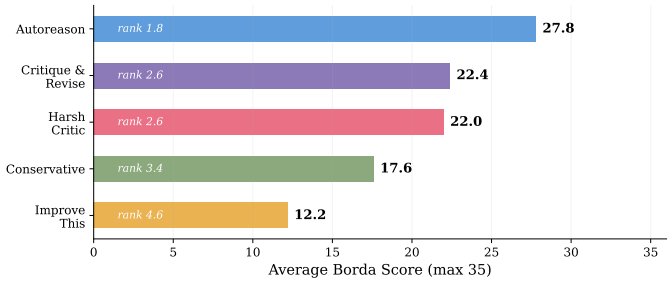

Figure 3. Aggregate performance. Autoreason (27.8 avg Borda, rank 1.4) never placed below 2nd.

4.2 Qualitative Improvement

The initial Task 1 output was a generic startup playbook: $49/user pricing, $100K MRR target with a 3-person team, no customer validation. After 14 passes, the converged output contained quantified pain points, team-based pricing matching the buying motion, customer validation details, competitive positioning against named tools, and unit economics. The adversarial process replaced vague aspirational claims with internally consistent specifics. These details are LLM-generated and not verified against real-world data; the improvement is in internal coherence and specificity, not factual accuracy.

4.3 Convergence and Trajectory

All 5 tasks converged at threshold k=2, requiring 9–28 passes (Figure 4). Policy tasks converge faster (10 passes) than multi-stakeholder operational processes (28 passes).

Figure 5 shows the Task 1 trajectory. At pass 15, A had won twice consecutively. Eleven more passes followed. A 7-judge panel comparing pass 15 vs pass 25 chose pass 15 by 6–1. Post-convergence passes degraded quality in this case.

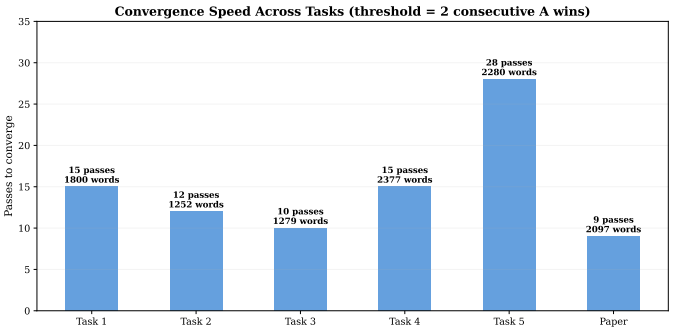

Figure 4. Passes to convergence across all tasks. Task 5 required the most (28); the paper (9) the least.

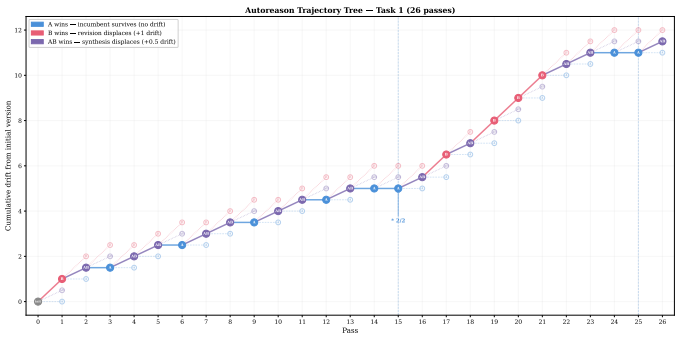

Figure 5. Trajectory tree, Task 1 (26 passes). Stars mark convergence points.

4.4 Chain-of-Thought Judges

Adding "think step by step about each version" to the judge prompt, with no architecture changes, dramatically improves convergence (Table 3). CoT judges produce scores of 8–9 for A wins (vs baseline's 6–7), acting as a debiasing mechanism: judges must articulate specific strengths before committing to a ranking, preventing verbosity bias and pattern-matching.

Judge Variant	Task 1	Task 2
Baseline (holistic)	14–15 passes	12 passes
Chain-of-thought	**5 passes**	**8 passes**
Decomposed (3 specialists)	7 passes	—

Table 3. CoT reduces convergence 3× on Task 1 (1.5× on Task 2). All subsequent experiments use CoT judges.

4.5 Robustness

Monte Carlo. Five independent runs of Task 1: 4/5 converged (80%). Convergence pass varied (6, 8, 14, 17) but final word counts clustered tightly (1451–1660), suggesting a consistent quality ceiling regardless of path (Figure 6).

Game-theoretic validation. Across 91 passes: only 1 Condorcet cycle (1.1%, near-perfect transitivity). Elo ratings plateau by pass 5–10. Pairwise dominance is fully transitive: autoreason > critique-and-revise > harsh_critic > conservative > improve_this. No rock-paper-scissors dynamics.

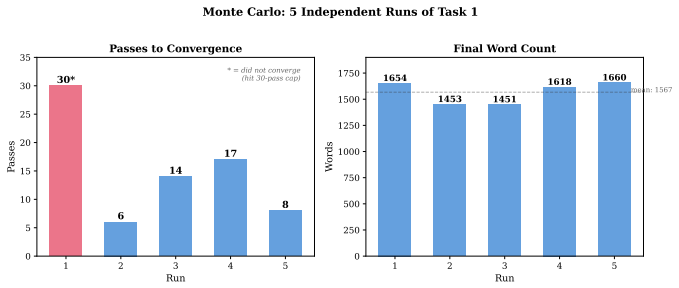

Figure 6. Monte Carlo: 5 runs of Task 1. Convergence speed varies; final output quality does not.

5 Model Scaling

We ran autoreason on Task 2 using claude-sonnet-4-6 (a stronger model) with CoT judges. The initial result was a complete reversal: autoreason came last (Table 4). Two rounds of remedy experiments then identified the root cause and a fix.

Method	Borda	1st Place
Critique & rev.	**31**	3
Improve this	30	4
Harsh critic	23	0
Conservative	14	0
Autoreason	7	0

Table 4. Sonnet 4.6, Task 2 (unconstrained). Autoreason ran 50 passes without converging; all baselines ran 15 passes.

Over 50 passes, A won only 6 times (12%). AB dominated with 30+ wins. The stronger model produced syntheses so consistently preferred by judges that the incumbent could never survive. This is structurally different from the bloat/prune oscillation with Sonnet 4: with 4.6, judges were decisive enough (scores of 8–9 for AB) to never let A through.

5.1 Round 1: Convergence Remedies

We tested four modifications (Sonnet 4.6, Task 2, CoT judges, 50-pass cap):

Remedy	Passes	A	AB	B	Conv.?
Margin (≥2 pts)	15	3	8	4	**Yes**
All combined	17	3	12	2	**Yes**
Scope-aware judges	35	4	25	6	No
Plateau detection	35	2	22	11	No
Baseline	*50*	*6*	*30+*	—	*No*

Table 5. Convergence remedies. Only the margin requirement (AB/B must win by ≥2 Borda) produces convergence.

The margin requirement converged at passes 15 and 17 (Figure 7). Scope-aware judges and plateau detection had no effect.

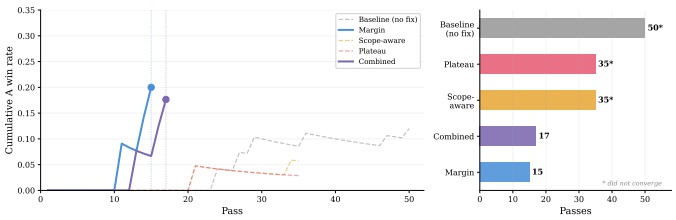

Figure 7. Left: cumulative A win rate (solid = converged). Right: passes to termination (* = cap).

But convergence ≠ quality. A 7-judge panel showed margin-converged output still places 4th–5th (Table 6). The margin rule solves termination but not quality; the incumbent accumulates synthesis drift across 15–17 passes of near-constant displacement.

Method	Borda (/49)	1st
Critique & rev.	**43**	2
Improve this	42	4
Harsh critic	39	1
Autoreason (combined)	26	0
Autoreason (margin)	23	0
Conservative	12	0
Autoreason (baseline)	11	0

Table 6. Margin recovers convergence but not quality. Autoreason still loses to all active baselines.

5.2 Round 2: Root Cause

Four evaluation-side modifications plus a scope-constrained task (500-word startup pitch from 8 fixed facts):

Modification	Passes	A	AB	B	Conv.?
Constrained task	**10**	**4**	**6**	**0**	**Yes**
Anchored judges	25	4	12	9	No
Subtractive synth.	17	0	10	7	No
Anchored + subtractive	17	1	6	10	No
Verdict hierarchical [11]	15	1	12	2	No

Table 7. Only the constrained task converges. Evaluation-side modifications, including hierarchical judge verification, do not.

We also tested hierarchical judge verification inspired by Verdict [11]: each judge's ranking is reviewed by a verifier that checks whether the reasoning reflects genuine quality differences or just length/detail preference, repeated 3 times with Borda aggregation. This produced 1 A win in 15 passes (7%), worse than the baseline's 12%. The verifier consistently confirmed the original judge's preference for AB, indicating that the per-pass preference is correct locally; the problem is trajectory-level drift that no single-pass evaluation can detect.

The constrained task converged at pass 10 with A scoring 9, the strongest convergence with Sonnet 4.6 in any experiment. Word counts stayed within 480–694 (vs 1895–2700 unconstrained). B never won a single pass.

A 7-judge blind panel confirmed quality (Table 8):

Method	Borda (/35)	1st
Autoreason	**30**	**3**
Improve this	27	2
Conservative	25	2
Harsh critic	13	0
Critique & rev.	10	0

Table 8. Constrained task: autoreason wins with Sonnet 4.6. Critique-and-revise expanded to 932 words, violating the 500-word constraint; autoreason stayed at 632.

The ranking inverts (Figure 8). The constrained task penalizes exactly the drift that autoreason's architecture prevents.

6 Competitive Programming

To validate the scope and structured-reasoning findings beyond subjective domains, we tested autoreason on competitive programming, where correctness is objectively verifiable via test cases. We sampled 150 problems from CodeContests [13] (50 easy, 50 medium, 50 hard by Codeforces rating) and compared three strategies at a matched compute budget of 6 LLM calls per problem (claude-sonnet-4).

Strategies. (1) *Single pass*: 6 independent attempts, return the best (pass@6). (2) *Critique-and-revise*: generate once, then 5 rounds of test-informed critique and revision. (3) *Autoreason*: generate once, then a structured analysis pass (problem analysis, approach analysis, failure analysis, alternative approaches, edge cases), followed by 4 rounds of reason-informed revision. Each revision round sees test results from public (visible) test cases. Final evaluation uses private (hidden) test cases to measure generalization.

Strategy	Public	Private	Easy	Med	Hard
Autoreason	**85%**	**64%**	44/50	29/50	23/50
Critique & rev.	81%	58%	35/50	31/50	21/50
Single pass	79%	59%	42/50	19/50	21/50

Table 9. Private-test pass rate (hidden tests, measuring generalization). Autoreason leads by 5–6 points overall. Private columns show per-tier counts out of 50.

Recovery is the mechanism. All three strategies solve roughly the same number of problems on the first attempt (∼83/150). The difference emerges from what happens after initial failure. Among 53 problems where both autoreason and single-pass failed on the first attempt, autoreason recovered 33 (62%) vs. single-pass's 23 (43%). A paired McNemar's test on the discordant pairs (15 recovered only by autoreason vs. 5 recovered only by single-pass) is significant (p=0.041). The effect size is medium (Cohen's h=0.32).

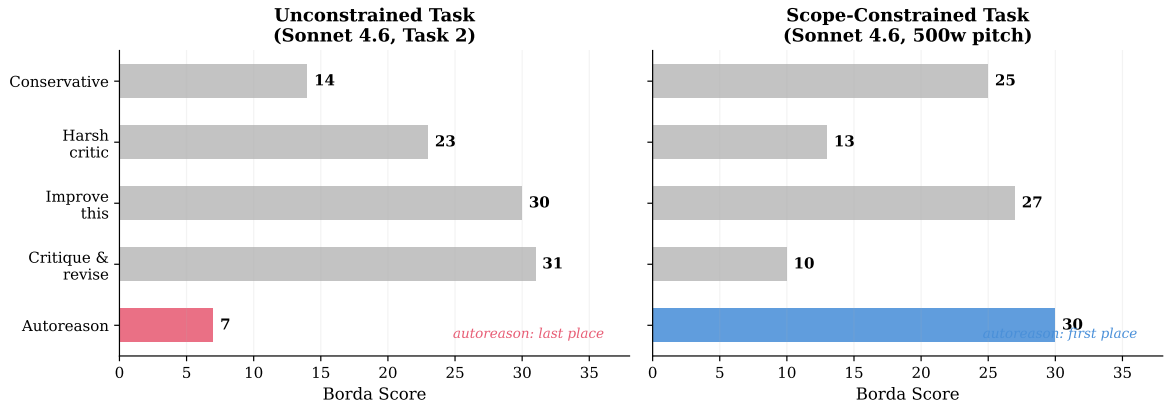

Figure 8. Same model (Sonnet 4.6), same method, opposite results. Left: unconstrained task, autoreason last. Right: scope-constrained task, autoreason first.

The structured analysis step—where the model reasons about *why* the approach failed before attempting a fix—is what drives this. Critique-and-revise sees the same test feedback but jumps directly to revision; it pattern-matches on error messages rather than reasoning about the underlying algorithmic problem.

Overfitting signal. Critique-and-revise shows the highest overfitting rate: 25% of problems pass public tests but fail private tests, compared to 22–23% for the other strategies. On hard problems specifically, critique-and-revise overfits 16/50 (32%) vs. autoreason's 16/50 and single-pass's 13/50. The refinement loop optimizes for visible examples at the cost of generalization when reasoning is shallow.

Connection to scope. Competitive programming problems are naturally scope-constrained: bounded input, specified output format, finite solution space. The test suite provides the external verification signal. These are exactly the conditions identified in Section 5 as necessary for autoreason to work. The code domain confirms the finding in a setting where quality is objective, not dependent on LLM judge preferences.

7 Writing This Paper

We ran autoreason on this paper using claude-opus-4 with 3 Opus judges and ground-truth context (actual experimental data). Converged in 9 passes.

Ground-truth critic. Without ground-truth access, the initial Opus generation hallucinated a fabricated ablation study, fake confidence intervals, wrong model names, and incorrect role descriptions (Figure 9). With ground-truth access, the critic caught all four on pass 1.

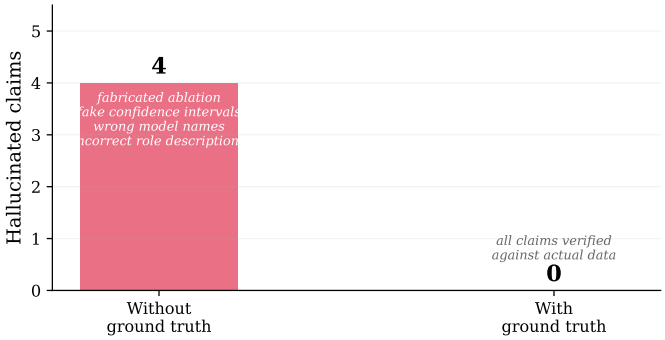

Figure 9. Hallucinated claims without ground truth (4) vs. with it (0).

Judge panel integrity. An earlier mixed panel (Opus + Sonnet + Gemini) failed to converge for 11+ passes because Gemini's output failed our parser, reducing the panel to 2 judges. Fixed to 3 working judges, the same incumbent converged in 2 passes (Figure 10). A broken judge does not add noise; it prevents equilibrium.

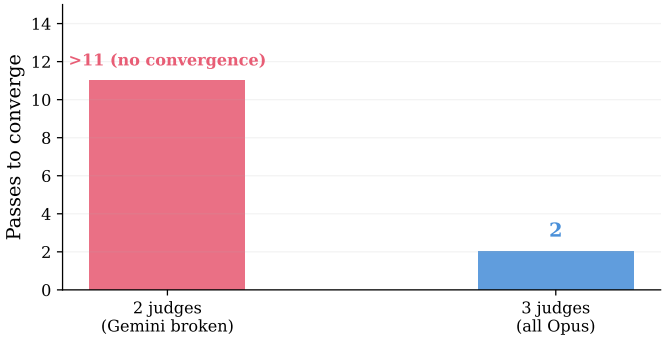

Figure 10. Broken parser: >11 passes, no convergence. 3 working judges: 2 passes.

8 Related Work

Autoresearch [1] uses val_bpb as an objective fitness function; autoreason extends this beyond objective metrics.

SlopCodeBench [2] shows iterative code degradation that prompting can't fix. **ACE** [3] documents context collapse in iterative rewriting. **Self-Refine** [4] demonstrates single-agent iterative improvement but Huang et al. [5] show self-correction is unreliable without external feedback; our code experiment (Section 6) quantifies this: critique-and-revise overfits to visible test cases while structured reasoning generalizes. **AI Safety via Debate** [6] grounds adversarial evaluation: truth has a natural advantage when agents argue before a judge. **Constitutional AI** [7] uses principle-based self-evaluation as training signal. **LLM-as-Judge** [8] documents systematic biases motivating our randomization and panel design. **LLM Council** [9] uses multi-model panels for single-turn evaluation; autoreason adds iteration with role separation. **PSRO** [10] provides game-theoretic framing for iterative strategy generation, loosely analogous to our pass structure. **Verdict** [11] scales judge-time compute through hierarchical verification and debate-aggregation; we tested its rank-then-verify pattern and found it does not address the scope problem (Section 5). **CodeContests** [13] provides competitive programming problems with hidden test cases; we use it to validate autoreason in an objectively verifiable domain.

9 Discussion

Three conditions for reliable self-refinement. Across both domains, autoreason works when: (1) scope is constrained (500-word pitch, competitive programming problem), (2) external verification exists (judge panel, test suite), and (3) reasoning is structured rather than reactive. Remove any one and the method degrades: unconstrained scope causes drift (Section 5), self-evaluation without external signal is unreliable [5], and shallow critique-and-revise overfits to visible feedback (Section 6).

Recovery, not first-attempt improvement. Autoreason does not help models get things right the first time. Its value is in what happens after initial failure: structured analysis of *why* the approach failed, consideration of alternatives, then targeted revision. In competitive programming, this recovery mechanism is statistically significant (p=0.041). In writing, it manifests as the judge panel rejecting regressions that accumulate unchecked in baselines.

CoT judges should be the default. The cheapest improvement: 3× faster convergence on Task 1 (1.5× on Task 2), more decisive scores, no architecture changes.

Scope is the central variable. The bloat/prune oscillation with Sonnet 4 (Figure 11) signals underdetermined scope. With stronger models, this oscillation becomes one-directional drift. Scope constraints eliminate it entirely, restoring both convergence and quality (Section 5). The competitive programming results confirm this in an objective domain: naturally scoped problems with test-suite verification are exactly where autoreason excels.

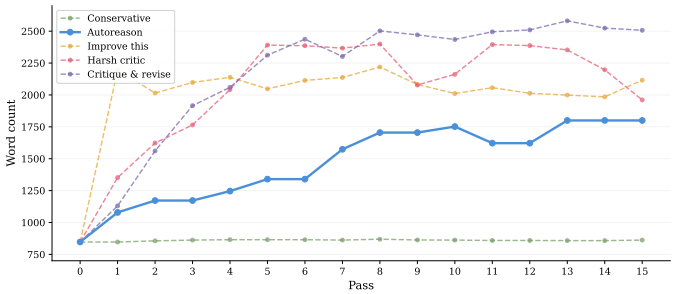

Figure 11. Word count trajectories (15 passes, same start). Autoreason (solid) grows controllably; baselines stagnate or bloat.

Limitations. All results come from one model family (Anthropic). Writing evaluation uses LLM judges, not human raters; the quality claims rest on LLM preferences. The code experiment provides objective verification but the overall private-test pass rate difference (64% vs. 58%) is not statistically significant at $p<0.05$; only the paired recovery analysis reaches significance. The constrained writing task is a single task (500-word pitch); the code domain provides broader validation (150 problems) but both use the same model. Scaling remedy experiments each ran once. Monte Carlo n=5 is underpowered.

10 Conclusion

Self-refinement is unreliable in general, but it works under identifiable conditions. We find three: constrained scope, external verification, and structured reasoning. Autoreason exploits all three. In subjective writing, it ranked first or second across 5 tasks with Sonnet 4 and first on a scope-constrained task with the stronger Sonnet 4.6. In competitive programming, it recovers significantly more failed solutions than alternatives (p=0.041), with the strongest gains when the model must reason about algorithmic failures rather than pattern-match on error messages.

The practical recommendation: pair iterative refinement with explicit scope constraints and external feedback signals, and invest the refinement budget in structured analysis of failures rather than additional generation attempts. The broader insight: the obstacle to iterative LLM self-improvement is not model capability but the absence of grounding. Without a signal external to the model's own reasoning loop, refinement is an echo chamber. With one, it converges.

References

[1] A. Karpathy. Autoresearch, 2024.

[2] G. Orlanski et al. SlopCodeBench: Benchmarking how coding agents degrade over iterative tasks. *arXiv:2603.24755*,

2026.[1]

[3] J. Zhang et al. ACE: Agentic context engineering. *ICLR*, 2026.

[4] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. *NeurIPS*, 2023. *arXiv:2303.17651*.

[5] J. Huang et al. Large language models cannot self-correct reasoning yet without external feedback. *arXiv:2310.01798*, 2023.

[6] G. Irving, P. Christiano, D. Amodei. AI safety via debate. *arXiv:1805.00899*, 2018.

[7] Y. Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv:2212.08073*, 2022.

[8] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023. *arXiv:2306.05685*.

[9] LLM Council, 2025.

[10] M. Lanctot et al. A unified game-theoretic approach to multiagent reinforcement learning. *NeurIPS*, 2017.

[11] R. Nair et al. Verdict: Scalable judge-time compute via hierarchical verification. *arXiv:2502.18018*, 2025.

[12] K. Arrow. *Social Choice and Individual Values*. Wiley, 1951.

[13] Y. Li et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[1] References [2] and [3] carry 2026 publication dates. These are correct and reflect recent preprints available at the time of submission.

A Prompts

A.1 Critic

```
System: You are a critical reviewer. Your only job is to find real problems. Be specific
and concrete. Do not suggest fixes.
User: Find real problems with this proposal. Focus on: things that won't work as
described, complexity that doesn't pay for itself, assumptions that are wrong, missing
pieces. Do NOT propose fixes. Just the problems.
```

A.2 Author B

```
System: You are a senior consultant revising a proposal based on specific criticisms.
Address each valid criticism directly. Do not make changes not motivated by an identified
problem.
User: [TASK] + [VERSION A] + [CRITIC OUTPUT] Revise to address these problems. For each
change, state which problem it fixes.
```

A.3 Synthesizer

```
System: You are given two versions as equal inputs. Take the strongest elements from each
and produce a coherent synthesis. This is not a compromise.
User: [TASK] + [VERSION X] + [VERSION Y] (labels randomized)
```

A.4 Judge (Baseline)

```
System: You are an independent evaluator. You have no authorship stake in any version.
User: [TASK] + Three proposals (randomized labels and order). Rank best to worst.
RANKING: [best], [second], [worst]
```

A.5 Judge (Chain-of-Thought)

```
System: You are an independent evaluator. Think carefully before deciding.
User: [TASK] + Three proposals. For each, think step by step: 1. What does it get
right? 2. What does it get wrong or miss? 3. Are numbers and claims defensible? 4. Is
detail appropriate or bloated? After reasoning, rank all three. RANKING: [best], [second],
[worst]
```

A.6 Baselines

```
Conservative: "You are improving a document. Make minimal improvements while preserving
what works. Do not add new sections or significantly expand scope." + [TASK] + [CURRENT
DOCUMENT]
Improve this: "Improve this document." + [TASK] + [CURRENT DOCUMENT]
Harsh critic: "You are a demanding reviewer and rewriter. Critically evaluate this
document and rewrite it, fixing all weaknesses you identify." + [TASK] + [CURRENT DOCUMENT]
Critique & revise: Step 1: "Produce a structured critique of this document. List specific
weaknesses." + [TASK] + [CURRENT DOCUMENT]. Step 2: "Revise the document to address each
criticism." + [TASK] + [CURRENT DOCUMENT] + [CRITIQUE]
```

B Trajectory Trees

Figure 12 shows trajectory trees for all experiments.

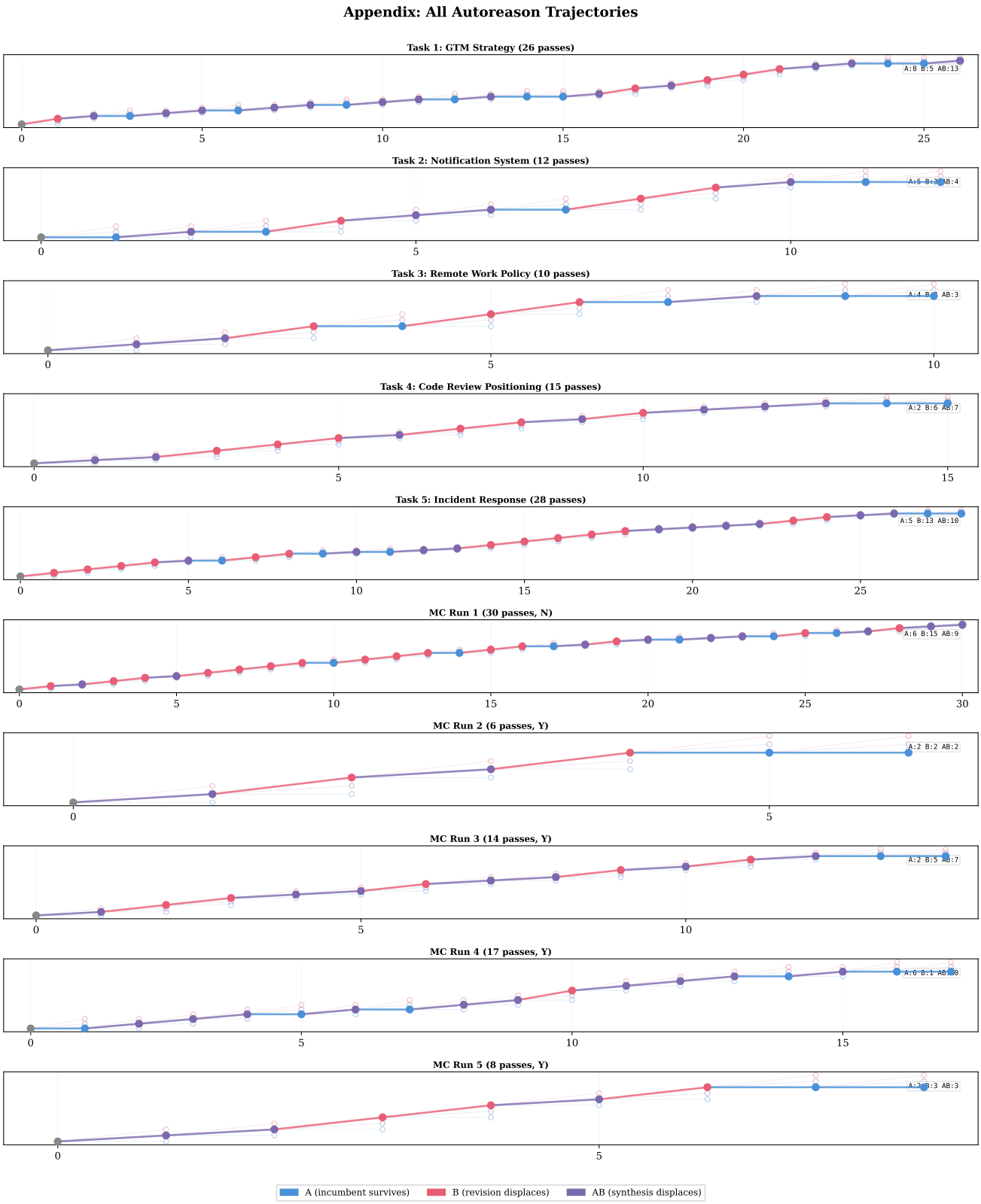

Figure 12. All trajectories. Tasks 2–3 converge quickly (10–12 passes). Task 5 requires 28 passes. Monte Carlo runs show variable convergence speed (6–30 passes) on the same task.